

Hands-on 1 Report: Create a Spring Web Project Using

Maven Objective

The objective of this hands-on was to create a Spring Boot web application using Maven, understand the project structure, configure the required dependencies, and successfully run the application using the embedded Tomcat server.

Tools and Technologies Used

- Eclipse IDE
 - Java 21
 - Spring Boot 3.x
 - Maven
 - Spring Initializr
 - Spring Web
 - Spring Boot DevTools
-

Procedure

Step 1: Create the Project

A new Spring Boot project was created using **Spring Initializr** with the following configuration:

- **Project:** Maven
- **Language:** Java
- **Group:** com.cognizant
- **Artifact:** spring-learn
- **Packaging:** Jar
- **Java Version:** 21

The following dependencies were added:

- Spring Web
- Spring Boot DevTools

The project was downloaded as a ZIP file and extracted.

Step 2: Import the Project

The extracted project was imported into Eclipse using:

File → Import → Maven → Existing Maven Projects

After importing, Maven automatically downloaded all the required dependencies.

Step 3: Understanding the Project Structure

The project contains the following important folders:

src/main/java

This folder contains the application's source code, including controllers, services, repositories, entities, and the main Spring Boot application class.

src/main/resources

This folder stores configuration files such as application.properties, static resources, and templates.

src/test/java

This folder contains unit and integration test classes used for testing the application.

Step 4: SpringLearnApplication.java

The main class generated by Spring Boot is:

```
@SpringBootApplication
public class SpringLearnApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringLearnApplication.class, args);
    }

}
```

The main() method serves as the entry point of the application. The SpringApplication.run() method initializes the Spring Application Context, performs component scanning, configures beans, starts the embedded Tomcat server, and launches the application.

Step 5: Understanding @SpringBootApplication

The @SpringBootApplication annotation combines three important annotations:

- @Configuration – Marks the class as a source of bean definitions.
 - @EnableAutoConfiguration – Automatically configures Spring Boot based on project dependencies.
 - @ComponentScan – Scans the package for Spring components such as controllers, services, and repositories.
-

Step 6: Understanding pom.xml

The pom.xml file is the Maven Project Object Model (POM) file. It manages project configuration, dependencies, plugins, and build information.

Important dependencies included in the project are:

- **spring-boot-starter-web** – Provides Spring MVC, REST support, and the embedded Tomcat server.
- **spring-boot-devtools** – Enables automatic application restart during development.
- **spring-boot-starter-test** – Provides testing support using JUnit and Mockito.

The spring-boot-maven-plugin is used to package and run the Spring Boot application.

Step 7: Running the Application

Initially, the application failed because port **8080** was already in use. The server port was changed to **8081** by adding the following property in application.properties:

```
server.port=8081
```

After restarting the application, it started successfully.

Successful console output included:

- Tomcat started on port 8081
 - Spring Application Context initialized successfully
 - SpringLearnApplication started successfully
-

Learning Outcomes

Through this hands-on, I learned:

- How to create a Spring Boot project using Spring Initializr.
 - How to import a Maven project into Eclipse.
 - The purpose of the src/main/java, src/main/resources, and src/test/java folders.
 - The role of the SpringLearnApplication class.
 - The purpose of the @SpringBootApplication annotation.
 - The structure and importance of the pom.xml file.
 - How Maven manages project dependencies.
 - How to resolve a port conflict by configuring the server port.
 - How to successfully run a Spring Boot application using the embedded Tomcat server.
-

Conclusion

The Spring Boot web project was successfully created, configured, and executed using Maven and Eclipse. The project structure, Maven configuration, and Spring Boot startup process were understood in detail. The application was successfully deployed on the embedded Tomcat server running on port **8081**, providing a strong foundation for developing RESTful web applications using Spring Boot.

Screenshots



